

S01-Text to Audio

The screenshot displays a REST client interface with the following details:

- URL:** `https://api.groq.com/openai/v1/audio/speech`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "model": "canopylabs/orpheus-v1-english",
3   "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How
4     can I assist you today?",
5   "voice": "diana",
6   "response_format": "wav"
7 }
```
- Status:** `200 OK` (1.47 s, 402.08 KB)
- Response:** A large black rectangular area representing the audio file, with a playback control bar at the bottom showing `0:08 / 0:08`.
- Footer:** Includes a taskbar with application icons and a system tray showing `29°C Mostly clear` and the date `5:33 AM 6/20/2026`.

"model": "canopylabs/orpheus-arabic-saudi",

GenAI-June-HomeAssignment / S01-Text to Audio Copy

POST https://api.groq.com/openai/v1/audio/speech

Send

Docs Params Authorization Headers (13) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "model": "canopylabs/orpheus-arabic-saudi",
3   "input": "[Cheerful] كيف يمكنني مساعدتك اليوم؟",
4   "voice": "aisha",
5   "response_format": "wav"
6 }
```

Body Cookies (1) Headers (23) Test Results 200 OK 843 ms 218.64 KB Save Response

Hex Preview Visualize

0:04 / 0:04

Runner Start Proxy Cookies Vault Trash 5:39 AM 6/20/2026

S02- MultiLingual

Overview | POST S02- MultiLingual | POST S01-Text to Audio | +

GenAI-June-HomeAssignment / S02- MultiLingual | Save | Share |

POST | https://api.groq.com/openai/v1/chat/completions | Send

Docs | Params | Authorization | Headers (11) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
11    }
12  ]
13 }
```

Body | Cookies (1) | Headers (21) | Test Results | 200 OK | 3.87 s | 1.77 KB | Save Response

{ JSON | Preview | Visualize

```
9   "message": {
10     "role": "assistant",
11     "content": "நன்றி! இந்த API-ஐ பயன்படுத்துவதற்கு பின்வரும் படிகளை பின்பற்றவும்:\n\n1. **பதிவு செய்தல்**: முதலில், நீங்கள் இந்த API-ஐ பயன்படுத்துவதற்கு ஒரு கணக்கை உருவாக்க வேண்டும். இதற்கு, நீங்கள் இந்த API-ஐ வழங்கும் வலைத்தளத்தில் பதிய வேண்டும்.\n2. **API கிளையன்ட் ID**: பதிவு செய்த பிறகு, நீங்கள் ஒரு API கிளையன்ட் ID பெறுவீர்கள். இந்த ID ஐ நீங்கள் இந்த API-ஐ பயன்படுத்துவதற்கு தேவைப்படும்.\n3. **API எண்ட்பாயின்ட்களைக் கண்டறிதல்**: இந்த API-ஐ பயன்படுத்துவதற்கு, நீங்கள் இந்த API-ஐ வழங்கும் வலைத்தளத்தில் உள்ள API எண்ட்பாயின்ட்களைக் கண்டறிய வேண்டும். இம் எண்ட்பாயின்ட்கள் உங்களுக்கு இந்த API-ஐ பயன்படுத்துவதற்கு தேவையான தகவல்களை வழங்கும்.\n4. **API கோரிக்கைகளை அனுப்பதல்**: இந்த API எண்ட்பாயின்ட்களைக் கண்டறிந்த பிறகு, நீங்கள் இந்த API-ஐ பயன்படுத்துவதற்கு தேவையான கோரிக்கை அனுப்ப வேண்டும். இந்த கோரிக்கைகள் பொதுவாக HTTP முறைகள் (GET, POST, PUT, DELETE) மூலம் அனுப்பப்படும்.\n5. **பதிவு பகுப்பாய்தல்**: இந்த API-ஐ பயன்படுத்துவதற்கு அனுப்பிய கோரிக்கைக்கு பதிலாக, நீங்கள் ஒரு பதிலைப் பெறுவீர்கள். இந்த பதில் பொதுவாக JSON அல்லது XML வடிவத்தில் இருக்கும். நீங்கள் இந்த பதிலை பகுப்பாய்த்து, தேவையான தகவல்களை பெறலாம்.\n\nஇந்த படிகளை பின்பற்றுவதன் மூலம், நீங்கள் இந்த API-ஐ வெற்றிகரமாக பயன்படுத்தலாம். மேலும், நீங்கள் இந்த API-ஐ பயன்படுத்தவாருக தேவையான தகவல்களை பெறவாருக நீங்கள் இந்த API ஐ வழங்கும் வலைத்தளத்தில் உள்ள வலைப்பதிவுகளைப் படிக்கலாம்."
```

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
    }
  ]
}
```

```
}
```

Response:

```
{
```

```
"id": "chatcmpl-6524572d-1737-4541-8d76-ceb36f706d32",
```

```
"object": "chat.completion",
```

```
"created": 1781915283,
```

```
"model": "llama-3.3-70b-versatile",
```

```
"choices": [
```

```
{
```

```
"index": 0,
```

```
"message": {
```

```
"role": "assistant",
```

```
"content": "நன்றி! இந்த API-ஐ பயன்படுத்துவதற்கு பின்வரும்  
படிகளை பின்பற்றவும்:\n\n1. **பதிவு செய்தல்**: முதலில், நீங்கள் இந்த API-  
ஐ பயன்படுத்துவதற்கு ஒரு கணக்கை உருவாக்க வேண்டும். இதற்கு,  
நீங்கள் இந்த API-ஐ வழங்கும் வலைத்தளத்தில் பதிவு செய்ய வேண்டும்.\n2.  
**API கிளையன்ட் ID**: பதிவு செய்த பிறகு, நீங்கள் ஒரு API கிளையன்ட் ID  
பெறுவீர்கள். இந்த ID ஐ நீங்கள் இந்த API-ஐ பயன்படுத்துவதற்கு  
தேவைப்படும்.\n3. **API எண்ட்பாயிண்ட்களைக் கண்டறிதல்**: இந்த API-ஐ  
பயன்படுத்துவதற்கு, நீங்கள் இந்த API-ஐ வழங்கும் வலைத்தளத்தில் உள்ள  
API எண்ட்பாயிண்ட்களைக் கண்டறிய வேண்டும். இந்த  
எண்ட்பாயிண்ட்கள் உங்களுக்கு இந்த API-ஐ பயன்படுத்துவதற்கு  
தேவையான தகவல்களை வழங்கும்.\n4. **API கோரிக்கைகளை  
அனுப்புதல்**: இந்த API எண்ட்பாயிண்ட்களைக் கண்டறிந்த பிறகு, நீங்கள்  
இந்த API-ஐ பயன்படுத்துவதற்கு தேவையான கோரிக்கைகளை அனுப்ப  
வேண்டும். இந்த கோரிக்கைகள் பொதுவாக HTTP முறைகள் (GET, POST, PUT,  
DELETE) மூலம் அனுப்பப்படும்.\n5. **பதிவை பகுப்பாய்வுத்தல்**: இந்த API-ஐ  
பயன்படுத்துவதற்கு அனுப்பிய கோரிக்கைக்கு பதிலாக, நீங்கள் ஒரு  
பதிவைப் பெறுவீர்கள். இந்த பதில் பொதுவாக JSON அல்லது XML
```

வடிவத்தில் இருக்கும். நீங்கள் இந்த பதிலை பகுப்பாய்ந்து, தேவையான தகவல்களை பெறலாம்.\n\nஇந்த பதிகளை பின்பற்றுவதன் மூலம், நீங்கள் இந்த API-ஐ வெற்றிகரமாக பயன்படுத்தலாம். மேலும், நீங்கள் இந்த API-ஐ பயன்படுத்துவதற்கு தேவையான தகவல்களை பெறுவதற்கு, நீங்கள் இந்த API-ஐ வழங்கும் வலைத்தளத்தில் உள்ள ஆவணங்களையும், எடுத்துக்காட்டுகளையும் பார்க்கலாம்."

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.056634676,
  "prompt_tokens": 133,
  "prompt_time": 0.006601974,
  "completion_tokens": 1679,
  "completion_time": 3.762856131,
  "total_tokens": 1812,
  "total_time": 3.769458105
},
"usage_breakdown": null,
"system_fingerprint": "fp_dae98b5ecb",
"x_groq": {
  "id": "req_01kvh6tqwwfz0av0ngy6wxjzj0",
  "seed": 171127502
},
"service_tier": "on_demand"
```

}

Chinese :

GenAI-June-HomeAssignment / S02- MultiLingual Copy

POST

https://api.groq.com/openai/v1/chat/completions

Send

DocsParamsAuthorizationHeaders (11)BodyScriptsTestsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

SchemaBeautify

```
2      "model": "llama-3.3-70b-versatile",
3      "messages": [
4        {
5          "role": "system",
6          "content": "You are an API Specialist. Always respond in the same language as the user."
7        },
8        {
9          "role": "user",
10         "content": "解释降雨形成的原因"
11        }
12      ]
13    }
14  }
15  "logprobs": null.
```

BodyCookies (1)Headers (21)Test Results

200 OK2.14 s1.9 KB

Save Response

JSON

PreviewVisualize

```
7      {
8        "index": 0,
9        "message": {
10         "role": "assistant",
11         "content": "降雨是地球上水循环过程中的一个重要环节，它的形成主要有以下几个原因：\n\n### 1. 水蒸发\n太阳的热量使得海洋、湖泊、河流、土壤和其他水体中的水分子从液态转化为蒸汽，升入空气中。这种过程称为蒸发。
\n\n### 2. 凝结\n当这些水蒸气上升到高空时，温度逐渐降低，水蒸气遇冷后开始凝结成小的水滴或者冰晶。这一过程形成了云。凝结的过程通常需要一个核，如尘埃粒子、盐粒等，水蒸气可以在这些核上凝结成微小的水滴。
\n\n### 3. 降水形成\n随着越来越多的水蒸气凝结到这些云滴上，云滴变得越来越重。当这些云滴无法再悬浮在空气中时，它们就会下落到地面，这一过程被称为降水。降水可以以多种形式出现，如雨、雪、冻雨、霰子等。
\n\n### 4. 大气条件\n降雨的形成还取决于大气的条件，如温度、空气压力、风向等。大气中的上升气流可以促进水蒸气的凝结过程，而风可以促进云层的移动和分布。
\n\n因此，降雨的形成是水蒸发、凝结、云滴聚集以及大气条件等因素综合作用的结果。这些因素共同作用，影响着降雨的强度、时间和区域分布。"
12        },
13        "logprobs": null.
```

RunnerStart ProxyCookiesVaultTrash

{

```
"model": "llama-3.3-70b-versatile",

"messages": [

  {

    "role": "system",

    "content": "You are an API Specialist. Always respond in the same language as the user."

  },

  {

    "role": "user",

    "content": "解释降雨形成的原因"
```

```
}  
]  
}
```

Response :

```
{  
  "id": "chatcmpl-3e5a7742-c2f3-4113-91e4-c5ff0d2b6a66",  
  "object": "chat.completion",  
  "created": 1781915465,  
  "model": "llama-3.3-70b-versatile",  
  "choices": [  
    {  
      "index": 0,  
      "message": {  
        "role": "assistant",
```

```
        "content": "降雨是地球上水循环过程中的一个重要环节，它的形成主要有以下几个原因：\n\n#### 1. 水蒸发\n太阳的热量使得海洋、湖泊、河流、土壤和其他水体中的水分子从液态转化为蒸汽，升入空气中。这种过程称为蒸发。\\n\\n#### 2. 凝结\n当这些水蒸气上升到高空时，温度逐渐降低，水蒸气遇冷后开始凝结成小的水滴或者冰晶。这一过程形成了云。凝结的过程通常需要一个核，如尘埃粒子、盐粒等，水蒸气可以在这些核上凝结成微小的水滴。\\n\\n#### 3. 降水形成\n随着越来越多的水蒸气凝结到这些云滴上，云滴变得越来越重。当这些云滴无法再悬浮在空气中时，它们就会下落到地面，这一过程被称为降水。降水可以以多种形式出现，如雨、雪、冻雨、雹子等。\\n\\n#### 4. 大气条件\n降雨的形成还取决于大气的条件，如温度、空气压力、风向等。大气中的上升气流可以促进水蒸气的凝结过程，而风可以促进云层的移动和分布。\\n\\n因此，降雨的形成是水蒸发、
```

凝结、云滴聚集以及大气条件等因素综合作用的结果。这些因素共同作用，影响着降雨的强度、时间和区域分布。"

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.048716796,
  "prompt_tokens": 58,
  "prompt_time": 0.001974904,
  "completion_tokens": 378,
  "completion_time": 2.048554692,
  "total_tokens": 436,
  "total_time": 2.050529596
},
"usage_breakdown": null,
"system_fingerprint": "fp_dae98b5ecb",
"x_groq": {
  "id": "req_01kvh70c5pee2t6wykwc53k9hm",
  "seed": 992384392
},
"service_tier": "on_demand"
}
```

S03-GenerateAPITestcasesFromSwaggerCode

GenAI-June-HomeAssignment / S03-GenerateAPITestcasesFromSwaggerCode

POST https://api.groq.com/openai/v1/chat/completions

Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```

1 {
2   "model": "openai/gpt-oss-120b",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API
7         schema-driven test generation."
8     },
9     {
10      "role": "user",
11      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java
12      Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful
13      class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI
14      setup."
15    }
16  ]
17 }

```

Body Cookies (1) Headers (21) Test Results 200 OK 3.09 s 2.67 KB Save Response

{ JSON Preview Visualize

```

12 {
  "reasoning": "We need to output Java code only, no markdown fences. Should include imports, class with @BeforeClass for
    baseURI. Use RestAssured. Use TestNG annotations. Need to create request bodies, maybe using Map or POJO. For
    simplicity, use a Map to construct JSON. Also need to store petId across methods; use static long petId.\n\nWill
    generate test class PetApiTests.\n\nMethods: createPet, getPetById, updatePet, deletePet.\n\nUse given/when/then
    pattern.\n\nAdd assertions: status code, response fields (e.g., id, name, status). Use JsonPath to extract.\n\nUse
    RestAssured.baseURI in @BeforeClass.\n\nAdd imports: io.restassured.RestAssured, io.restassured.http.ContentType, io.
    restassured.response.Response, org.testng.Assert, org.testng.annotations.BeforeClass, org.testng.annotations.Test,
    java.util.HashMap, java.util.Map.\n\nPotentially need to set basePath? Use full path.\n\nPetstore base URL: https://
    petstore.swagger.io/v2\n\nEndpoints:\nPOST /pet\nGET /pet/{petId}\nPUT /pet\nDELETE /pet/{petId}\n\nWill set static

```

```

{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest
Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore
APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include
explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful
class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and
assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8)

```

Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"

```
}  
]  
}
```

Response

```
{  
  "id": "chatcmpl-9d650597-4db4-4898-9820-ebae79a47e91",  
  "object": "chat.completion",  
  "created": 1781915607,  
  "model": "openai/gpt-oss-120b",  
  "choices": [  
    {  
      "index": 0,  
      "message": {  
        "role": "assistant",  
        "content": "import io.restassured.RestAssured;\nimport  
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport  
org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport  
org.testng.annotations.Test;\n\nimport java.util.Collections;\nimport  
java.util.HashMap;\nimport java.util.Map;\n\npublic class PetApiTests {\n\n    private static final  
String BASE_URI = \"https://petstore.swagger.io/v2\";\n    private static long petId;\n    private
```

```

static final String INITIAL_NAME = "Rex";\n private static final String UPDATED_NAME =
"Max";\n private static final String PET_STATUS_AVAILABLE = "available";\n private static
final String PET_STATUS_SOLD = "sold";\n\n @BeforeClass\n public void setUp() {\n
RestAssured.baseURI = BASE_URI;\n }\n\n @Test(priority = 1)\n public void createPet() {\n
petId = System.currentTimeMillis();\n Map<String, Object> pet = new HashMap<>();\n
pet.put("id", petId);\n pet.put("name", INITIAL_NAME);\n pet.put("photoUrls",
Collections.singletonList("http://example.com/photo")); \n pet.put("status",
PET_STATUS_AVAILABLE);\n\n Response response = RestAssured.given()\n
.contentType(ContentType.JSON)\n .body(pet)\n .when()\n
.post("/pet")\n .then()\n .extract()\n .response();\n\n
Assert.assertEquals(response.getStatusCode(), 200);\n
Assert.assertEquals(response.jsonPath().getLong("id"), petId);\n
Assert.assertEquals(response.jsonPath().getString("name"), INITIAL_NAME);\n
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_AVAILABLE);\n
}\n\n @Test(priority = 2, dependsOnMethods = "createPet")\n public void getPetById() {\n
Response response = RestAssured.given()\n .pathParam("petId", petId)\n
.when()\n .get("/pet/{petId}")\n .then()\n .extract()\n
.response();\n\n Assert.assertEquals(response.getStatusCode(), 200);\n
Assert.assertEquals(response.jsonPath().getLong("id"), petId);\n
Assert.assertEquals(response.jsonPath().getString("name"), INITIAL_NAME);\n
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_AVAILABLE);\n
}\n\n @Test(priority = 3, dependsOnMethods = "getPetById")\n public void updatePet()
{\n Map<String, Object> petUpdate = new HashMap<>();\n petUpdate.put("id",
petId);\n petUpdate.put("name", UPDATED_NAME);\n petUpdate.put("photoUrls",
Collections.singletonList("http://example.com/photo")); \n petUpdate.put("status",
PET_STATUS_SOLD);\n\n Response response = RestAssured.given()\n
.contentType(ContentType.JSON)\n .body(petUpdate)\n .when()\n
.put("/pet")\n .then()\n .extract()\n .response();\n\n
Assert.assertEquals(response.getStatusCode(), 200);\n
Assert.assertEquals(response.jsonPath().getLong("id"), petId);\n
Assert.assertEquals(response.jsonPath().getString("name"), UPDATED_NAME);\n
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_SOLD);\n }\n\n
@Test(priority = 4, dependsOnMethods = "updatePet")\n public void deletePet() {\n
Response deleteResponse = RestAssured.given()\n .pathParam("petId", petId)\n
.when()\n .delete("/pet/{petId}")\n .then()\n .extract()\n
.response();\n\n Assert.assertEquals(deleteResponse.getStatusCode(), 200);\n
Assert.assertTrue(deleteResponse.asString().contains(String.valueOf(petId)));\n\n Response
getAfterDelete = RestAssured.given()\n .pathParam("petId", petId)\n

```

```
.when()\n        .get(\"/pet/{petId}\")\n        .then()\n        .extract()\n        .response();\n\n    Assert.assertEquals(getAfterDelete.getStatusCode(), 404);\n    }\n}",
```

"reasoning": "We need to output Java code only, no markdown fences. Should include imports, class with @BeforeClass for baseURI. Use RestAssured. Use TestNG annotations. Need to create request bodies, maybe using Map or POJO. For simplicity, use a Map to construct JSON. Also need to store petId across methods; use static long petId.\n\nWill generate test class PetApiTests.\n\nMethods: createPet, getPetById, updatePet, deletePet.\n\nUse given/when/then pattern.\n\nAdd assertions: status code, response fields (e.g., id, name, status). Use JsonPath to extract.\n\nUse RestAssured.baseURI in @BeforeClass.\n\nAdd imports: io.restassured.RestAssured, io.restassured.http.ContentType, io.restassured.response.Response, org.testng.Assert, org.testng.annotations.BeforeClass, org.testng.annotations.Test, java.util.HashMap, java.util.Map.\n\nPotentially need to set basePath? Use full path.\n\nPetstore base URL: https://petstore.swagger.io/v2\n\nEndpoints:\nPOST /pet\nGET /pet/{petId}\nPUT /pet\nDELETE /pet/{petId}\n\nWill set static long petId.\n\nIn createPet: generate random petId (System.currentTimeMillis()) and name \"Rex\". Build map: id, name, photoUrls (list), status.\n\nUse Map<String, Object> pet = new HashMap<>(); pet.put(\"id\", petId); pet.put(\"name\", \"Rex\"); pet.put(\"photoUrls\", Collections.singletonList(\"url\")); pet.put(\"status\", \"available\");\n\nNeed import Collections.\n\nAfter POST, assert status 200, response id equals petId.\n\nStore petId static for later.\n\nIn getPetById: GET, assert status 200, id matches, name matches.\n\nIn updatePet: change name to \"Max\", status \"sold\". Use same map but modify fields, send PUT. Validate status 200 and new fields.\n\nIn deletePet: DELETE, assert status 200. Optionally GET after deletion expecting 404.\n\nWill include try-catch? Not needed.\n\nAdd @Test(priority) to enforce order.\n\nLet's craft code.\n\n"

```
    },\n\n    "logprobs": null,\n\n    "finish_reason": "stop"\n  }\n],\n\n"usage": {\n\n    "queue_time": 0.279781423,\n\n    "prompt_tokens": 396,\n\n    "prompt_time": 0.015496066,
```

```

"completion_tokens": 1308,
"completion_time": 2.69869424,
"total_tokens": 1704,
"total_time": 2.714190306,
"completion_tokens_details": {
  "reasoning_tokens": 448
}
},
"usage_breakdown": null,
"system_fingerprint": "fp_803c0ba83d",
"x_groq": {
  "id": "req_01kvh74npdeqvsrc66682gvpw6t",
  "seed": 1922913282
},
"service_tier": "on_demand"
}

```

The screenshot shows the 'JSON Formatter & Toolkit' web application. The 'JSON to Plain Text' tab is active. The 'INPUT JSON' field contains a complex JSON object with nested arrays and objects. The 'PLAIN TEXT' field shows the resulting plain text output, which is a Java code snippet using the RestAssured library to perform a DELETE request on a pet resource.

JSON Formatter & Toolkit

Paste any JSON — format, validate, minify, compare, and extract keys.

Format / Validate Minify Compare Key Extractor **JSON to Plain Text**

[Convert JSON to Plain Text](#) [Copy Output](#)

INPUT JSON

```

{
  "completion_tokens": 1308,
  "completion_time": 2.69869424,
  "total_tokens": 1704,
  "total_time": 2.714190306,
  "completion_tokens_details": {
    "reasoning_tokens": 448
  },
  "usage_breakdown": null,
  "system_fingerprint": "fp_803c0ba83d",
  "x_groq": {
    "id": "req_01kvh74npdeqvsrc66682gvpw6t",
    "seed": 1922913282
  },
  "service_tier": "on_demand"
}

```

PLAIN TEXT 4045 chars

```

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.junit.Assert;
import org.junit.BeforeClass;
import org.junit.Test;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

public class PetApiTests {

    private static final String BASE_URI =
        "https://petstore.swagger.io/v2";

```

Groq API Complete Guide — @Copyrights 2026 Testleaf -All Rights Reserved

```
import io.restassured.RestAssured;
```

```
import io.restassured.http.ContentType;

import io.restassured.response.Response;

import org.testng.Assert;

import org.testng.annotations.BeforeClass;

import org.testng.annotations.Test;


import java.util.Collections;

import java.util.HashMap;

import java.util.Map;


public class PetApiTests {


    private static final String BASE_URI = "https://petstore.swagger.io/v2";

    private static long petId;

    private static final String INITIAL_NAME = "Rex";

    private static final String UPDATED_NAME = "Max";

    private static final String PET_STATUS_AVAILABLE = "available";

    private static final String PET_STATUS_SOLD = "sold";


    @BeforeClass

    public void setUp() {

        RestAssured.baseURI = BASE_URI;

    }


    @Test(priority = 1)

    public void createPet() {
```

```
petId = System.currentTimeMillis();
Map<String, Object> pet = new HashMap<>();
pet.put("id", petId);
pet.put("name", INITIAL_NAME);
pet.put("photoUrls", Collections.singletonList("http://example.com/photo"));
pet.put("status", PET_STATUS_AVAILABLE);
```

```
Response response = RestAssured.given()
    .contentType(ContentType.JSON)
    .body(pet)
    .when()
    .post("/pet")
    .then()
    .extract()
    .response();
```

```
Assert.assertEquals(response.getStatusCode(), 200);
Assert.assertEquals(response.jsonPath().getLong("id"), petId);
Assert.assertEquals(response.jsonPath().getString("name"), INITIAL_NAME);
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_AVAILABLE);
}
```

```
@Test(priority = 2, dependsOnMethods = "createPet")
public void getPetById() {
    Response response = RestAssured.given()
        .pathParam("petId", petId)
```

```
.when()  
.get("/pet/{petId}")  
.then()  
.extract()  
.response();
```

```
Assert.assertEquals(response.getStatusCode(), 200);  
Assert.assertEquals(response.jsonPath().getLong("id"), petId);  
Assert.assertEquals(response.jsonPath().getString("name"), INITIAL_NAME);  
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_AVAILABLE);  
}
```

```
@Test(priority = 3, dependsOnMethods = "getPetById")  
public void updatePet() {  
    Map<String, Object> petUpdate = new HashMap<>();  
    petUpdate.put("id", petId);  
    petUpdate.put("name", UPDATED_NAME);  
    petUpdate.put("photoUrls", Collections.singletonList("http://example.com/photo"));  
    petUpdate.put("status", PET_STATUS_SOLD);
```

```
Response response = RestAssured.given()  
    .contentType(ContentType.JSON)  
    .body(petUpdate)  
    .when()  
    .put("/pet")  
    .then()
```



```

        .extract()
        .response();

Assert.assertEquals(response.getStatusCode(), 200);
Assert.assertEquals(response.jsonPath().getLong("id"), petId);
Assert.assertEquals(response.jsonPath().getString("name"), UPDATED_NAME);
Assert.assertEquals(response.jsonPath().getString("status"), PET_STATUS_SOLD);
}

```

```

@Test(priority = 4, dependsOnMethods = "updatePet")

```

```

public void deletePet() {

```

```

    Response deleteResponse = RestAssured.given()

```

```

        .pathParam("petId", petId)

```

```

        .when()

```

```

        .delete("/pet/{petId}")

```

```

        .then()

```

```

        .extract()

```

```

        .response();

```

```

Assert.assertEquals(deleteResponse.getStatusCode(), 200);

```

```

Assert.assertTrue(deleteResponse.asString().contains(String.valueOf(petId)));

```

```

Response getAfterDelete = RestAssured.given()

```

```

    .pathParam("petId", petId)

```

```

    .when()

```

```

    .get("/pet/{petId}")

```

```
.then()  
.extract()  
.response();
```

```
Assert.assertEquals(getAfterDelete.getStatusCode(), 404);
```

```
}
```

```
}
```